Atomic Use Case as a Concept to Support the MDE Approach to Web Application Development

Kinh Nguyen[1] **and Tharam Dillon**[2]

[1] Computer Science & Computer Engineering Department, La Trobe University,Australia. kinh.nguyen@latrobe.edu.au

[2] Faculty of Information Technology, University of Technology Sydney,Australia. tharam@it.uts.edu.au

Abstract: While use case is a popular technique for capturing requirements, the process of proceeding from use cases to subsequent modeling activities is to a large extent still unclear. In this paper, we propose the concept of atomic use case to facilitate this process. In particular we demonstrate how to construct a precise model for the business logic layer and to establish clear relationships between business logic model to other models such as the domain model, user interface model, navigation model, and the business process model. We also explore how the atomic use case concept can be incorporated into UWE (UML Web Engineering) Methodology.

Keywords: Use Case, Atomic Use Case, Business Logic, Domain Model, Navigation Model, Business process Model.

1 Introduction

In this paper, we propose the concept of atomic use case as a fundamental concept to support the MDE (Model-Driven Engineering) approach to web-based application development.

A web application, in particular a web information system, can be seen as having the following major components: (a) web-based components, (b) business logic and data source components, (c) other components such as web-services, portlets, agents, metadata, data mining etc. We will be largely concerned with the first two kinds of components.

The concept of atomic use [6] is proposed as a solution to what we regard as a long-standing problem of information system, be it relational or object-oriented, traditional and otherwise. We are referring to the problem of analyzing and modeling the behavior of information systems. It will be useful to consider it first from the angle of relational database and object-oriented information system.

One often-claimed advantage of relational database technology is the separation of the static aspect and the dynamic aspect (known as "data independence"). The static aspect, which is about the structure of the database, has been handled in practice reasonably well (though there is room for improvement). In contrast, the dynamic aspect, which is about the operations of the database system, is handled quite poorly.

Broadly speaking, this problem has gone through four phases: from being an urgent problem to a forgotten problem to a brief revival of interest and then back to forgotten. In the 70s, IFIP (The International Federation on Information Processing) recognized it as a key problem and organized a number conferences, inviting researchers to propose solutions to this problem. A number of proposals were made but none of them were adequate to the task, especially as a practical solution. Consequently, the problem gradually slipped into the phase of being a forgotten problem. Interest in the problem was briefly revived in the late 80s with the emergence of Z notation. Before long, it was realized that Z notation could not deliver the expected results, and the the problem slipped back into the state of a forgotten problem. In the mean time, the industry handles the problem in an informal (hence ambiguous) and ad hoc manner, which incurs huge hidden costs (poor understanding of functional requirements, poor specification for programmers, etc.).

As for object-oriented information systems, first of all, we can observe that UML without OCL would be too imprecise to describe behavior of information systems. An activity diagram can represent at a number of tasks and the flows between them; but it lacks the facilities to describe the tasks in detailed and precise terms. The same is true of sequence diagrams, which can show the sequence of messages in a collaboration,

but are not good at describing the detailed effects of such messages. Though there are situations in which the sequence of messages alone can be very helpful (e.g. to explain how enterprise Java beans work, that is, how a client, through a number of stubs, can communicate with the beans on the server), it is not the case for information systems. For information systems, it is the effects of the messages that really tell us what really is going on. In fact, it is more often than not that after drawing pages and pages of sequence diagrams, when it comes to implementation, we have to ask a whole host of questions all over again. The gap between sequence diagrams and the implementation code is far too big in most cases. In the language of MDE, the model does not have sufficient information for the subsequent transformation act.

To strengthen the models, OCL may well provide a stepping stone toward a solution. There are, however, a couple of important issues to investigate. First, does OCL have adequate expressive power for specifying complex applications? OCL does not have, for example, a notation to denote the common mechanism of message sending. Second, suppose OCL has, or can be made to have, sufficient expressive power, how are we to use it? For example, what would be the units of behavior that we are going to use it for? And how are we going to identify these units in practice?

We propose atomic use case as an answer to the problem of precise behavior modeling. In this paper, we will introduce the concept and demonstrate how we can use it to construct a precise model for the business logic layer, and from there, to establish clear relationships between business logic model to other models such as the domain model and the user interface. Finally, we explore how the atomic use case concept can be incorporated into UWE (UML Web Engineering) Methodology and briefly describe how we may apply this concept to the Travel Agency case study.

2 Introductory Example

We will take a example to show what atomic use cases are and the role they play through out the life-cycle. Because we wish to show the role this concept can play in the whole life-cycle, we will need to choose a very simple example.

Problem Statement. Consider an application in which we are required to maintain information about a set of employees. Each employee has a unique ID, a name, and a phone number. The ID and name of an employee cannot be changed. System operations,

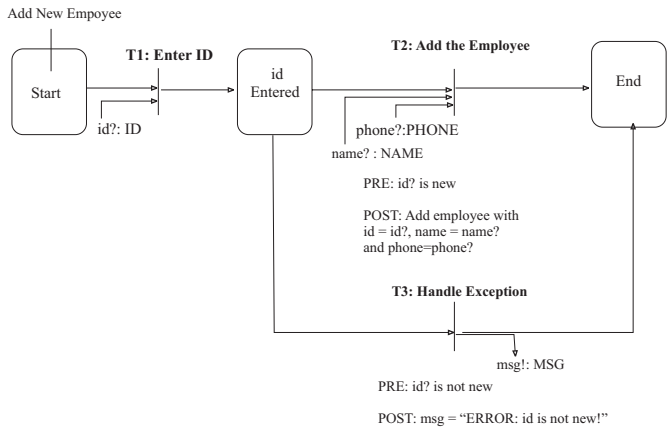

Figure 1: A Petri Net Representation for "Add an Employee" Use Case.

or use cases, include: (1) Add an employee, (2) Delete an employee, (3) Change the phone number of an employee, (4) Retrieve employees by ID, by name, or by phone number.

We can go through the use cases one by one, identify the atomic use cases, and formally specify them. Through this process, as will be shown, we end up with the complete model to capture full functionality of the application.

2.1 Identify and Specify Atomic Use Cases

Consider the operation (use case) to "Add an Employee." A use case description for this operation can be as follows:

"To add a new employee, first, the user enters the id of the new employee. The system checks to determine if the id is new. If it isn't, an error message will be displayed and the operation is terminated. Otherwise, the user enters next the name and the phone number.The system then creates a new employee with the input data and saves it."

The use case and the various path it can take can be represented by a high-level Petri net shown in Figure 1 (What we present here is a simple version of what we call the "obligation net" [7], a high-level Petri in which each transition has pre- and postconditions to precisely express the obligations the system has to fulfill.)

We now make a shift in the way we view the system's behavior. Instead of thinking in terms of "do this then do that" or "if this, then do that" (the procedural view), we consider the operation as one whole unit

and ask "For this use case, in how many different ways can the system makes its response?" (the declarative view). We can observe that for the current use case, the system can respond in two ways, corresponding to two paths in the system obligation net.

The first path consists of transitions T1 and T2. Following this path, the system makes a *positive* response: it adds a new employee to the information base (precondition permitted). The second path consists of transitions T1 and T3. In this case, the system makes a *do-nothing* response: it simply leaves the system in its prior state due to the non-fulfillment of the precondition.

We refer to the response described by the first path as an *atomic use case*, and the one described by the second path as an *exception use case*.

Here is a definition:

Definition: *An atomic use case is conceived as an* indivisible *response by the system that either (1) effects a change of the system's state, which takes the system from a consistent state to a consistent state, to reflect an event taking place in the application domain, or (2) performs a query that is of interest to the user in its own right.*

Back to the use case at hand, having identified the atomic use case, and in keeping with the "atomic" viewpoint, we can specify it (informally) in terms of the input, output, pre- and postconditions as follows:

- Input: id?, name?, phone?
- Output: None
- Pre: id? is new
- Post: Create a new employee with id?, name?, phone? and add the employee set

We can now seek to specify the atomic use case formally. As expected, the specification always follows a consistent format which consists of input, output, pre- and postconditions. Some of these elements may be absent in a particular case. For the "Add Employee" introduced earlier can be specified as follows:

AddNewEmployee

input

$id? : ID$
$name? : NAME$
$phone? : PHONE$

pre

$id? \notin \{e : allEmployees \bullet p.id\}$

post

$\exists\, newEmployee : allEmployees' \bullet$
$\quad newEmployee \notin allEmployees$
$\quad newEmployee.id = id?$
$\quad newEmployee.name = name?$
$\quad newEmployee.phone = phone?$
$\quad allEmployees' = allEmployees \cup \{newEmployee\}$

Two points should be made from this example. First, in order to precisely specify the use case, we need to use some formal notation. In the above example, we have used Object-Z [2].

Second, we have assumed the existence of two classes. One is the class *Employee* (only the static features are required at this stage):

Employee

$id : ID$
$name : NAME$
$phone : PHONE$

...

The other is a system class. It represents the system from the functional view point. It maintains a set of employees and provides method to manage that set of employees. We will call the class *EmployeeApp*:

EmployeeApp

$allEmployees : \mathbb{P}\, Employee$

$\forall\, e1, e2 \bullet e1 \neq e2 \Leftrightarrow e1.id \neq e2.id$

...

In this example, the *Employee* class is actually the whole of the domain model. In general, to formally specify the atomic use case, we need (a) the domain model or part of the domain model relevant to the use case, and (b) the system class. One crucial aspect of the system class is that through the attributes of the system object we can get to all the domain objects of the system.

In other words, to formally specify the atomic use cases, we need to perform some domain analysis. We do not need to cover the whole of the application domain model. It is sufficient to cover only part of the application domain relevant to use cases at hand. In this domain analysis, we only need to be concerned with the static aspect. The dynamic aspect will be discovered and formal specified by our work on atomic use cases.

We can go through the use cases one by one, and identify and formally specify the atomic use case for

each of them. The collection of atomic use case specifications thus obtained is *a complete function requirements model* of the application.

2.2 Derive Methods for Domain and System Classes to Obtain a Complete Business Logic Model

Once an atomic use case is formally specified, we can deduce the methods required of the domain classes and the system class to support that use case.

Each canonical use case will become a method of the system class. For the "Add a New Employee" use case, we require the method shown below in the system class:

EmployeeApp

$allEmployees : \mathbb{P}\, Employee$

$\forall\, e1, e2 \bullet e1 \neq e2 \Leftrightarrow e1.id \neq e2.id$

AddEmployee

$id? : ID$
$name? : NAME$
$phone? : PHONE$

$id? \notin \{e : allEmployees \bullet e.id\}$
$\exists\, e : Employee \bullet$
$\quad e \notin allEmployees$
$\quad e.Init(id?/id?, name?/name?, phone?/phone?)$
$\quad allEmployees' = allEmployee \cup \{e\}$

...

From the postcondition of that method of the system class, it is clear that we need to be able to create new *Employee* instances. Thus, we have identified a method (constructor) required of the domain class to support the use case. The method is expressed in Object-Z as an *INIT* schema as shown below:

Employee

$id : ID$
$name : NAME$
$phone : PHONE$

Init

$id? : ID$
$name? : NAME$
$phone? : PHONE$

$id = id?$
$name = name?$
$phone = phone?$

...

By repeating this activity for all atomic use cases, we would get a *complete business logic layer model* as far as the functional requirements are concerned. In general, this model consists of a system class and all the domain classes relevant to the particular application.

2.3 Implement and Test the Business Logic Model

Having derived the methods required of the system and domain classes, we can implement or prototype them. An implementation for the "Add New Employee" use case is shown below. We need one method for the system class:

```
public void addNewEmployee( String id, String name,
String phone) throws Exception
{
  // compute the precondition.
  // Assume that collectIds() returns the set of ids
  // of a set of employees
  boolean pre = allEmployees.collectIds().contains(id);

  // if precondition is not satisfied,
  // abort the operation
  if (! pre)
  {   throw new Exception( "The ID already exists!" );
  }

  // otherwise, create the new employee and add it
  // to the information base
  Employee newEmployee = new Employee(id, name, phone);
  allEmployees.add( newEmployee );
}
```

and one method for the domain class *Employee*:

```
public Employee(String id, String name, String phone)
{
    this.id = id;
    this.name = name;
    this.phone = phone;
}
```

Note that we have all the details from the business logic model we need to do that. No further discovery activities are needed.

Testing. Once the two methods above are available, we can test them with a testing script such as the one shown below. The variable name *theSystem* is to emphasize the fact that an instance of *EmployeeApp* represents the complete system in this example.

```
// create a system object
EmployeeApp theSystem = new EmployeeApp();

// add first employee and display the the system's state
// (toString completely displays the system's state)
theSystem.addNewEmployee("E10", "Smith", "1234");
System.out.println( theSystem.toString() );
```

```
// add second employee and display the the system's state
theSystem.addNewEmployee("E20", "Adams", "2345");
System.out.println( theSystem.toString() );

// try to add employee with an existing id
// and observe that the system's state remains the same
theSystem.addNewEmployee("E10", "Clarke", "3456");
System.out.println( theSystem.toString() );
```

The testing script contains several test cases (scenarios of the current use case), which are based on the precondition specified in the atomic use case. For each test, the state of the system object is displayed to verify that the implementation satisfies the postcondition specified in the atomic use case.

The tests reveal two important consequences of the implementation of the atomic use case.

- First, we can perform the required operation, i.e. adding employees.
- Second, and just as important, the system can protect itself from invalid requests and preserves the integrity of its state.

The ability to make appropriate responses to both valid and invalid requests is, of course, exactly what we should be looking for. And quite significantly, note that we can achieve all of this by considering *only* the atomic use cases.

2.4 The Functional Core

Once we proceed to implement the rest of the operations specified in the business logic model, the two classes that we obtain is an executable component that allows us to perform all the required operations (add and delete employees, change phone numbers, etc.). We call this the *functional core* (to distinguish it from other components of a complete system such as the user interface, etc.).

In other words, we get a "basic core of the system" that is fully functional in the sense that this component can store the relevant information, update the information, and respond to queries in support of the business activities. For this reason, we regard the functional core to be the *business logic layer* of the system. That is, in our approach, we take the business logic layer to be precisely the implementation of the *atomic* use cases relevant to the application.

Moreover, once the functional core has been fully tested, we can build the desired graphical user interface as a separate layer on top of it. As will be shown, it is possible to build a separate GUI layer that interacts with the functional core only at a small number of well-defined points.

2.5 In Summary

If we

1. consider the use cases relevant to a system,
2. analyze each of those use cases and identify the atomic use cases associated with them, and
3. derive and implement those atomic use cases (as shown in the example above)

then we will obtain an executable component that can service all the valid requests and reject all the invalid ones. We call this executable component the functional core of the system, and we take it to be the "business logic layer" of the system in language of n-tier architecture.

Below, we will provide some clarification on the atomic use case concept.

3 Further Clarification on The Concept of Atomic Use Case

In the definition of atomic use case, presented earlier, the criterion that atomic use case must *"take the system from a consistent state to a consistent state"* is significant. The following simple example illustrates this point.

Example - Enroll Student. Consider the case of enrolling students in subjects. Suppose subjects are classified as *core* or *optional*, and each student must take at least 3 core subjects. Without the condition that a student must take at least 3 core subjects, the act of enrolling a student in a subject is an atomic use case. With that condition in place, that act is no longer an atomic use case: it may cause the information base to be in an inconsistent state. The atomic use use in this case must be to enroll a student in a set of subjects in one go. More precisely,

– The inputs are a student id and a set of subject codes

– The outputs: NONE (it is clearer not to regard error messages as output; they are implied by the preconditions)

– The preconditions are: (1) the id must exist, (2) each unit must exists, (3) the set contains at least three core unit

– Postcondition: Enroll the student in those subjects.

Similarly, the phrase *"to reflect an event taking place in the application domain"* provides a useful criterion for identifying atomic use case. The following example illustrates this point.

Example - Add Student or Staff. Consider an application which deals with students and staff in an academic institution (suppose we maintain some different information about them). "Add a Student" and "Add a Staff" are atomic use cases. In one of our presentations, it has been asked: Should we take "Add a Person" as an atomic use case? The answer is "No". In the application domain, we may have the event of "Having a new student" or "Having a new staff member", but not the event "Having a new person". "Person' is an abstraction with some information left out of student or staff, and the so-called event "Having a New Person" cannot fully describe the situation. Furthermore, it we consider the states of the information base, we can see this clearer: When the information base change from state S to state S', then S' may be S plus information about a new student or a new staff, but simply about a new person.

Finally, when the use case is a query use case it needs to be *"of interest to the user in its own right."* The following example serves to illustrate this point.

Example - Redistribution Parts between Warehouses. In [3], Jacobson presents a rather sophisticated screen to show how the user may interact with a system to redistributes parts among various warehouses (we move items from the 'From' warehouse to the 'To' warehouses). There is a drop-down list to select the 'From' warehouse. When a 'From' warehouse is selected by the user, the system respond by listing the rest of the warehouses as potential 'To' warehouses. In making this response, the system would need to perform a query against the information base. Now, it is unlikely that such a query is of interest to a user by itself (in this example, it serves as a small step in determining the potential 'To' warehouse and where in that warehouse we should move items to). If that is the case, the query does not amount to an atomic use case. It is simply a query that supports the user interface.

In our approach, we would extend the functional core to provide the user interface-support queries. The functional core and the extended part together are called the *extended function core* (Figure 2).

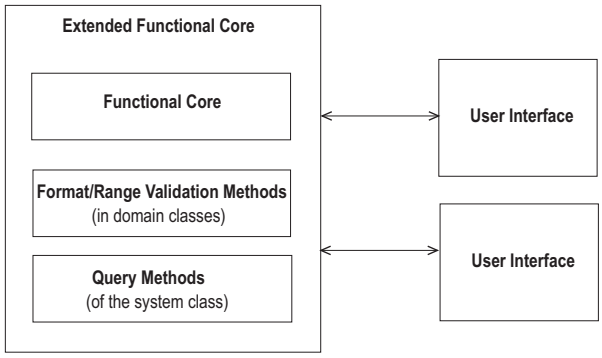

Figure 2: The Extended Functional Core to Support User Interface.

4 Consistent Relationships between the Structural and Behavioral Models

As shown in the introductory example, to formally specify the atomic use cases, we need to perform some domain analysis. We do not need to cover the whole of the application domain model. It is sufficient to cover only part of the application domain relevant to use cases at hand. In this domain analysis, we only need to be concerned with the static aspect. The dynamic aspect will be discovered and formal specified by our work on atomic use cases.

The fact that we need to perform the domain analysis is not a chore, it is in fact one of the strengths of the atomic use case approach. This activity establishes a precise relationship between the static model and the dynamic model. The specification of the atomic use case provides an opportunity to validate the static domain model. If there are things that are missing or incorrect in the domain model, we often discover them when we specify the atomic use cases.

Another advantage of the atomic use case approach is that it is possible to build the static model in an incremental, use case-by-use case basis, and integrate those partial model to form a complete domain model.

5 Atomic Use Cases and Graphical User Interfaces

As briefly shown earlier, we can identify atomic use cases through the descriptions of the 'normal' use cases. But, that is not the only option. In fact, it is more practical to do so through the graphical user interface.

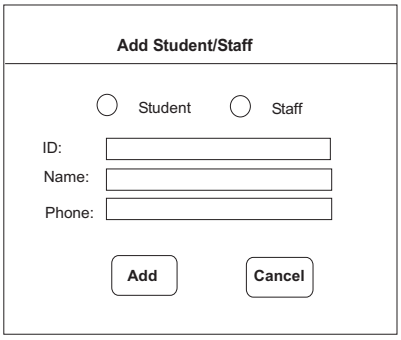

Figure 3: Screen to Add Student or Staff.

In the industry, they are less likely not talk about use cases; they often talk about user interface and how the user interact with the user interface. We will be talking the 'language of the industry' when we identify the atomic use cases through their user interface sketches (or design) and the description of how they work.

Example - Add Staff or Student Consider the screen in Figure 3.

– When the user checks either the 'Student' or the 'Staff' checkbox, no query is made to the information base. The user interface simply 'remembers' that the user has made that choice.

– When we enter id, the system would respond by checking whether the id is new or not, and warn us if it's not new. This action requires the system to make a query to the underlying information base. The purpose of this query is to support the user interface.

– After the user has entered relevant information and press button "ADD", the system will respond by calling the business layer to add a student or a staff member. Thus, out of this screen we have two atomic use cases: they are the two cases in which the information base may change its state.

In general, given a screen or a series of screens, web-based or otherwise, many events can be generated by the user's actions. By examining how the system responds to each of these events, in particular how it interacts with the information base, we can identify atomic use cases and all the queries to support the functioning of the user interface.

(Recall that the queries to support the user interface can be put in the extended part of the extended functional core. The extended functional core provides all the methods to construct a fully functional user interface.)

The relationships between the atomic use cases and the events generated by graphical user interfaces of any kind clearly indicate how atomic use cases can be applied to the analysis, modeling and specification of web applications.

In fact an event generated by the user action on a web-based interface can be specified as a contract (to be fulfilled by the method that handle the event on the server side). This contract consists of

– the input data (they come from the HTML form or cookies or the session object),

– the precondition (e.g. these data values must exist in the expected source), and

– the postcondition (updating of the cookies or session object, performing query or update action against the business logic layer, i.e. invoking atomic use cases, and generating the web page to send back to the browser).

The content of such a contract for a web event can be formally specified, especially with the availability of the atomic use cases.

6 Implementing the System Class

The system class can be implemented explicitly or implicitly. By "implemented explicitly, we mean there is a software component to which we can send messages to perform the atomic use case, one single message for each atomic use case.

Example - Prototype. Consider the employee list example. As a fully functional prototype in Java, for example, we can implement the *Employee* and *EmployeeApp* classes directly, making sure that they are serializable (to support persistence of course). We can then create an object of the *EmployeeApp* class and save it as an ObjectStream (to provide persistence, of course).

Then, in our web-based interface, we can present a HTML form to get the data about a new employee, and on the server-side (through servlets, or JSP, etc.), after successfully extracting the employee's details, we simply retrieve the *EmployeeApp* instance (let's call it *employeeApp* from the ObjectStream file, and send a message to *EmployeeApp*:

```
employeeApp.addEmployee(id, name, phone)
```

And that is how the web layer interacts with the business logic layer: only through *EmployeeApp* (we can also include in this class methods to support the user interface).

$\forall x : X \bullet p(x)$	`X size = (X select: [:p | p(x)]) size`
$\exists x : X \bullet p(x)$	`( X select: [:P | p(x) ] ) isEmpty not`
$x \in X$; $x \notin X$	`x in: X (or X includes: x )`
$x \notin X$	`x notIn: X (or (X includes: x) not)`
$X \cup \{a\}$	`X add: a`
$X \setminus \{a\}$	`X remove: a`
$X \cup Y$	`X union: Y`
$X \cap Y$	`X intersect: Y`
$X \setminus Y$	`X removeAll: Y`
$\{x : X \mid p(x)\}$	`X select: [ :x | p(x) ]`
$\{x : X \bullet e(x)\}$	`X collect: [ :x | e(x) ]`
$\exists x : X \bullet$ $p(x)$ $\blacktriangleright x.oper$	`x := X detect: [ :x | p(x) ].` `x oper`

(Note that `in:aCollection` can be defined as a method in the class `Object` as `aCollection includes: self`)

Figure 4: Equivalent Expressions between Formal Specification and Smalltalk

(The parameters should be primitive data types or string and collections of these, or collection of collections of basic data types and string, but none of user-defined objects.)

On Transformation Rules. Note that the *addEmployee* method is exactly what we presented earlier. To implement it, we simply take the formal specification and implement the pre- and postcondition specified therein. When we dwell into the details of the pre- and postconditions, we will see that in most cases we need to handle a small number of common expressions. These expressions are are list in Figure 4, which show the equivalents in Smalltalk. Equivalent Java code segments would be less concise, but definitely standard patterns can easily be established.

In the language of MDE, we have a formal platform independent model of the business logic layer (in this example, they are *Employee* and *EmployeeApp* classes, and we readily formulate transformation rules to transform it to target platform-specific component (in this case, the 'platform' is the basic J2SK).

Relational Implementation Consider the employee list example again. Suppose we are to use a relational database for persistence if we use JDBC, we can have a Java class called *EmployeeAppFnCore* (Employee Application Functional Core). This class is an explicit implementation of our system class. It has static methods with signatures such as

```
public static void addEmployee(String id, String name,
  String phone) throws Exception
```

Then, our server-side (through servlets, or JSP, etc.), after successfully extracting the employee's details, can simply make a call to *EmployeeAppFnCore*:

```
EmployeeAppFnCore.addEmployee(id, name, phone)
```

In this case, how do we go from the specification to the implementation. It depends on what we choose to model the atomic use cases. First, we can choose to do so with an object-oriented model (as shown in this paper). In this case, we need to convert that into a relational model. That is we convert the class diagram into a relational schema, and for each atomic use case, we express the pre- and postconditions as a method acting on the relational schema. The first task (obtaining the relational schema) is rather straight forward and many mapping rules have been suggested. The second task, at the intuitive level is very straight forward, but at the formal level seems to be quite hard to formulate (due to the mismatch between the object-oriented and the relational models).

Alternatively, we could start with a specification based on the relational model. In this case, standard Z notation is not quite suitable for practical use (as witnessed by history). We have experiment with an extension of Z. We call it RZ, 'R' for 'relational'. It does not have (yet) a formal semantics, though intuitively the meanings of the additional constructs are clear. As a kind of 'formal pseudo code', RZ works really well (quite unambiguously) in expressing the intentions of the modeler. For the consumption of those who do not want to use mathematical notations, we have added to few features to SQL to form a pseudo code language to express the atomic use cases (i.e. their pre- and postconditions) in a highly (but not formally) precise manner. We are currently investigating the use of an object-oriented specification language (such as Object-Z or OCL) for specifying pre- and postconditions against a relational schema by representing tuples of a relation as simple objects where all the attributes are publicly accessible for manipulation. The use of Maude [1] could be quite promising for this task as well, and we plan to look into it.

EJB Implementation We can implement each atomic use case as a session bean, which may access data sources in a distributed manner. In this case, the

system class is implemented implicitly as a collection of session beans.

However, in most cases, it is better to put all the atomic use cases in one session bean (one method for each atomic use case). This can be seen as an application of the facade pattern. In this case, the system class is implemented explicitly. Conceptually, such an implementation would be much clearer and easier to understand. It would make it easier for the client programs to locate and use the beans on the server-side : there is only one bean that the client should be aware of.

Note. It is not unusual to break up an application into a small number of major components. In such case cases, what we have so far referred to as "system class" will be used to represent the components and the atomic use cases are described in the context of a particular component. Also, we must specify the interactions among the components in terms of the messages they can send to each other.

7 Applying Atomic Use Cases to UWE

UWE (UML-based Web Engineering) [5] is a methodology for web application development with ArgoUWE [4] as a supporting CASE tool. One of the motivations for UWE is to provide support for complex business processes. Let us explore how we can apply the concept of atomic use case to UWE.

The example of an e-shop given in [4]. This example illustrates the basic steps and the basic models of UWE. The example shows four basic UWE models: the conceptual model, the navigation model, the (business) process structure model, and the process flow model.

Part of the navigation model for the e-shop example is reproduced in Figure 5. It shows the navigation nodes, process nodes, navigation links (between navigation nodes), and process links (between navigation nodes and process nodes). One of the process node is the "Checkout" node, whose details are shown in the process flow model in Figure 6.

From the atomic use case point of view, we would perceive this Checkout node as consisting of two "parts" which are associated to two different kinds of concerns. The first is about the business logic (which is captured by an atomic use case), and the second is about the user interface design.

The atomic use case in this example would be the one that takes details about an order submitted by the user (such as customer name, address, credit card number, whether they want it wrap, etc.) and updates the underlying information base of the e-shop accordingly. The update should leave the information base in a consistent state. More precisely, the only difference between the pre-state and post-state is the existence of a new order with all the information necessary for shipping and charging. For convenience, let us call this atomic use case "Take Order".

With the Take Order atomic use case in mind, we can see the role of the Checkout process flow model: its main aim is to obtain the details needed as input for the Take Order atomic use case.

Viewing it this way, the Checkout process flow model could be replaced by the specification of a user interface. Regardless of whether we model this behavioral "unit" as a process model or as a user interface, the ultimate aim is still to get the information required for the Take Order atomic use case.

We may argue that we should model this behavioral unit as a process model and use that model to design the user interface. However, this view is problematic in the sense that it could easily involve an over-specification. To see the over-specification, we can think of a graphical user interface which has two screen sections that the user can interact with in parallel: a section to set the payment method and a section to set the wrapping options (and buttons to cancel or to place order). Such a screen would allow us to get the information we need, but it does not constraint us to the flow pattern described in the Checkout process flow. In other words, the Checkout process flow is one way to get this information but it is not the only one.

The crucial point here, it could be argued, is the content of the information, not how we obtain it (for example, if we could get the all the relevant details about an order by email, that would be adequate as well). For that reason, we would replace the Checkout process flow by navigation node whose purpose is to collect information such as customer details, payment method and wrapping option (we may note that payment methods and wrapping options should be part of the domain model), and to pass that information on to the Take Order atomic use case. The Take Order atomic use case can be represented in the UWE model as a business process node. In this way, the process structure model would consists mainly of nodes that represent the atomic use cases. Note that some of the process nodes may represent external processes, which may be modeled by specifying inputs and outputs. Still some of the process nodes may be manual processes which should be modeled as part of the conceptual model.

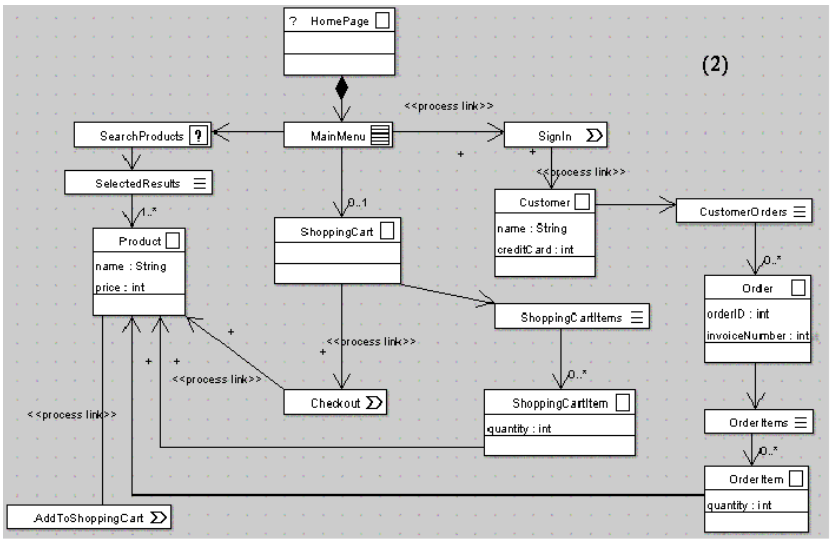

Figure 5: Navigation Model for the e-shop. Source: Knapp et al. [4]

Note that we still need the business process model. For example, we would want to capture the fact that following the acceptance of an order we should ship that order within two days, say. In this context, the use of atomic use cases can provide two main advantages. First, it provides a way to decide on the grade of granularity for the process nodes (which we take to be essentially the atomic use cases). Second, the content of the nodes can be unambiguously modeled in a consistent format (input, output, pre- and postconditions).

8 Atomic Use Cases and the Travel Agency Case Study

Consider the Travel Agency Case Study described in [8]. We will give a sketch as to how one may approach this case study using atomic use cases.

We will view the application as made up of the following components: the Personal Agent Assistant, the Broker Agent, the Transportation Company and the Finance Company. In describing these components, for a start, to concentrate on the key points, let us first make two simplifying assumptions that a trip requested by the customer is *not* to be broken up into several trips.

The Personal Agent Assistant (or Assistant for short) functions as an entity that process a request from the customer. Assuming that the Assistant handles one request at a time, we can think of it as a finite state machine with change-of-state events as atomic use cases.

To process a request, the Assistant maintains a list of Brokers and a Financial Companies, data about the customer bookings and payment details. It would also need to store the current request and the list of current offers. With this set-up, the Assistant would have the following main atomic use cases:

1. processRequest (to take a request from the customer and to send back the sorted list of offers). Input: a request; output: list of offers (which could be empty); pre: request is valid; post: store request AND store offers AND ensure that offers match the request

2. cancelRequest (to cancel the request). Input: none; output: none; pre: current offers are not empty; post: delete current request and offers

3. makeBooking (to take a booking from the user). Input: the offer identification, payment details; output: booking confirmation message; pre: the offer is one of the current offers; post: validate payment details with Financial Company AND send message to confirm the booking for the selected offer and cancel the rest.

The Agent Broker would maintain a list of Transportation Company. The main atomic use case it has

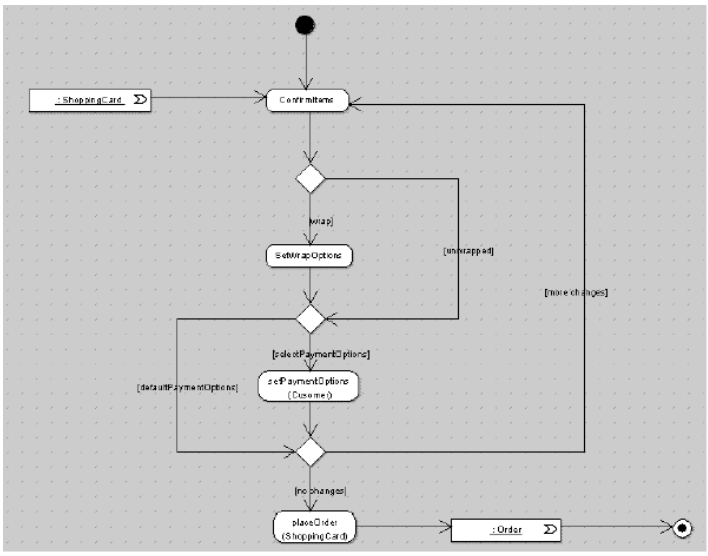

Figure 6: Navigation Model for the e-shop. Source: Knapp et al. [4]

are:

1. provideOffers (to take a request from the Assistant and to respond with a list of offer). Input: a request; output: a list of offers; pre: none; post: offers match request AND store the offers (together with the agent's if it works with more than one agents).

2. confirmBooking (to confirm the booking of an offer which were previously temporarily booked). Input: selected offer identification; output: confirm message; pre: selected offer is in the current list of offers; post: send message to confirm booking for selected offer and cancel the rest.

For the Transportation Company and the Finance Company, essentially we need to define the interface and the assumptions about the contracts related to the messages.

Having identified those atomic use cases, we can proceed to specify them formally, and construct the conceptual model (perhaps in the process of formalizing), the navigation model, and the business process model.

9 Conclusion

In this paper, we have introduced the concept of atomic use case. It is a natural concept and therefore easy to grasp. In fact, it has appeared in various guises in the systems development literature. However, to really benefit from it, we need to have a clear understanding of the concept. Toward this end, we provided a definition (one that aims to assist us in identifying the atomic use cases). We showed how we can identify the atomic use cases and specify them, and how we can build a complete business layer with them. Finally, we explored how we can incorporate the concept into UWE for web application development, which is likely to lead to a more definite choice of granularity for the process nodes and clearer relationships between the various models.

References

[1] M. Clavel, F. Duran, S. Eker, P. Lincoln, N. Marti-Oliet, J. Meseguer, and J. Quesada. Maude: specification and programming in rewriting logic. *Theoretical Comput. Sci.* , 285:187-243, Sept. 1995.

[2] R. Duke and G. Rose (2000) *Formal Object-Oriented Specification Using Object-Z*, MacMillan.

[3] Jacobson J., Ericsson M. and Jacobson P. (1992) *Object-Oriented Software Engineering: A Use Case Driven Approach*, Wokingham:Addison-Wesley.

[4] Alexander Knapp, Nora Koch, Gefei Zhang, and Hanns-Martin Hassler. Modeling Business Processes in Web Applications with ArgoUWE, In *7th International Conference on the Unified Modeling Language*

(UML2004), LNCS 3273, 69-83, Springer Verlag, October 2004.

[5] Nora Koch and Andreas Kraus. The expressive Power of UML-based Web Engineering. In *Second International Workshop on Web-oriented Software Technology (IWWOST02),* D. Schwabe, O. Pastor, G. Rossi, and L. Olsina, editors, CYTED, 105-119, June 2002.

[6] Kinh Nguyen, Tharam Dillon, Atomic Use Case: A Concept for Precise Modelling of Object-Oriented Information Systems, in *OOIS'03, The Ninth International Conference on Object-Oriented Information Systems,* Geneva, Switzerland, 2003

[7] Kinh Nguyen. *A Semi-Formal Object-Oriented Method for Analysis and Modelling of the Functional Requirements of Information Systems*, PhD Thesis in Computer Science, La Trobe University, 2003.

[8] A Travel Agency System, Case Study for Workshop on model-driven Web Engineering (MDWE 2005), http://www.lcc.uma.es/ av/mdwe2005/TheTAEexample